Malnad College of Engineering,Hassan

(An Autonomous Institution affiliated to VTU, Belgavi)

ACTIVITY REPORT

ON (23CS606B)

" Smart Inventory and Billing Management System"

Submitted in partial fulfillment of
the requirements for the award of the degree of

Bachelor of Engineering in
Computer Science and Engineering

Submitted by

Samruddhi H S	4MC23CS197
Anjali S H	4MC24CS401
Bhimappa	4MC24CS402
Jeevan Y R	4MC24CS408
Mohan S K	4MC24CS411

Submitted to

Mr. Ravi Kumar D

Assistant Professor

Department of Computer Science and Engineering

Malnad College of Engineering
Hassan - 573201, Karnataka, India

2025-2026

1. Problem Statement

Small and medium-scale retail businesses in India continue to rely on manual, paper-based methods for managing their inventory and billing processes. This approach leads to a series of operational inefficiencies and business losses that directly impact profitability and growth.

1.1 Identified Challenges

- Inventory mismanagement stock shortages and overstocking occur due to lack of real-time visibility.
- Error-prone manual billing hand-written receipts lead to calculation mistakes, incorrect GST computation, and customer disputes.
- No low-stock alerts businesses discover out-of-stock situations only at the point of sale, causing lost revenue.
- Absence of business analytics owners have no data-driven insight into daily sales trends, profit margins, or top-performing categories.
- GST compliance burden manually computing multi-rate GST (5%, 12%, 18%, 28%) for every transaction is time-consuming and error-prone.
- Fragmented records sales history is stored in paper registers, making audits and reconciliation extremely difficult.

1.2 Proposed Solution

This project delivers a comprehensive Smart Inventory and Billing Management System a full-stack web application that automates and centralises all inventory and billing operations. The system provides role-based secure access, real-time stock tracking, automated GST calculations, digital invoice generation with receipts, and a live analytics dashboard that enables data-driven business decisions.

2. System Architecture & Design

2.1 High-Level Architecture

The application follows a standard three-tier client–server architecture separated into a React.js Single-Page Application (SPA) on the frontend and a RESTful Node.js/Express.js API on the backend, backed by a MongoDB NoSQL database.

Tier	Technology	Responsibility
Presentation	React.js + Vite + Lucide Icons	UI rendering, navigation, user interaction
Application	Node.js + Express.js	Business logic, REST API, JWT auth
Data	MongoDB + Mongoose ODM	Persistent data storage and queries

2.2 Architectural Flow

Browser (React SPA) → HTTP / REST API calls → Express.js Server → Mongoose ODM → MongoDB

JWT Bearer tokens are included in every API request header for authentication. The backend validates the token using a shared JWT secret before processing any route logic.

2.3 Frontend Module Structure

```
frontend/src/
  ├── App.jsx              ← Root component & page router
  ├── context/AuthContext.jsx  ← JWT state + authFetch wrapper
  ├── components/Sidebar.jsx   ← Navigation sidebar
  └── pages/
      ├── Login.jsx            ← Authentication page
      ├── Dashboard.jsx        ← Analytics & KPI cards
      ├── Inventory.jsx        ← Product management
      ├── Categories.jsx       ← Category CRUD
      ├── Billing.jsx          ← POS billing interface
      └── SalesHistory.jsx     ← Invoice history & details
```

2.4 Backend Module Structure

```
backend/
  ├── server.js             ← Express app entry point
  ├── config/database.js    ← Mongoose schemas, models & seed data
  ├── middleware/auth.js    ← JWT verification middleware
  └── routes/
      ├── auth.js           ← Login & token verification
      ├── products.js       ← Product CRUD + low-stock endpoint
      ├── categories.js     ← Category CRUD
      ├── billing.js        ← Invoice creation & retrieval
      └── analytics.js      ← Dashboard KPIs & trend data
```

3. Database Schema & Models

MongoDB is used with Mongoose ODM. Four collections power the entire application: Users, Categories, Products, and Invoices.

3.1 User Model

Field	Type	Required	Notes
username	String	Yes	Unique — login identifier
password	String	Yes	bcrypt hashed (10 salt rounds)
name	String	Yes	Display name
role	String	No	Default: 'admin'

3.2 Category Model

Field	Type	Required	Notes
name	String	Yes	Unique category name
description	String	No	Optional description text

3.3 Product Model

Field	Type	Required	Notes
name	String	Yes	Product display name
sku	String	Yes	Unique auto-uppercased
category_id	ObjectId (ref)	No	References Category model
stock_quantity	Number	No	Default: 0
reorder_level	Number	No	Default: 10 triggers low-stock alert
purchase_price	Number	Yes	Cost price for profit calculation
selling_price	Number	Yes	Excl. GST
gst_rate	Number	No	Default: 18 (percentage)
created_at	Date	No	Auto-set on creation

3.4 Invoice Model

Field	Type	Required	Notes
invoice_number	String	Yes	Format:INV-YYYYMMDD-XXX
customer_name	String	Yes	Walk-in customer name
customer_phone	String	Yes	Contact number
subtotal	Number	Yes	Sum before GST
gst_amount	Number	Yes	Total GST collected
grand_total	Number	Yes	subtotal + gst_amount
payment_mode	String	No	Cash / Card / UPI default Cash
items	Array (Embedded)	Yes	Invoice line items sub-documents
created_at	Date	No	Auto-timestamp

Embedded Invoice Item Sub-document

Field	Type	Description
product_id	ObjectId	Reference to Product (for profit calc)
product_name	String	Snapshot of product name at sale time
quantity	Number	Units sold
unit_price	Number	Selling price excl. GST at sale time
gst_rate	Number	GST rate percentage
gst_amount	Number	GST collected for this line
total_price	Number	unit_price x qty + gst_amount

4. API Details

All endpoints are prefixed with /api. Protected routes require Authorization: Bearer <token> header.

4.1 Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/login	Public	Authenticate user; returns JWT token + user object
GET	/api/auth/verify	Private	Verify token validity; returns current user info

4.2 Category Endpoints

Method	Endpoint	Auth	Description
GET	/api/categories	Private	Get all categories sorted by name
POST	/api/categories	Private	Create new category (name required, unique)
PUT	/api/categories/:id	Private	Update category name/description
DELETE	/api/categories/:id	Private	Delete — blocked if products are linked

4.3 Product Endpoints

Method	Endpoint	Auth	Description
GET	/api/products	Private	List all products; supports ?search= and ?category_id= filters
GET	/api/products/low-stock	Private	Products where stock_quantity <= reorder_level
POST	/api/products	Private	Create product (SKU auto-uppercased, must be unique)
PUT	/api/products/:id	Private	Update product details; validates SKU uniqueness
DELETE	/api/products/:id	Private	Delete product by ID

4.4 Billing Endpoints

Method	Endpoint	Auth	Description
POST	/api/billing/invoice	Private	Create invoice; validates stock, deducts qty, generates INV number
GET	/api/billing/invoices	Private	List all invoices; supports ?search= on name/phone/number
GET	/api/billing/invoices/:id	Private	Get single invoice with full item breakdown

4.5 Analytics Endpoints

Method	Endpoint	Auth	Description
GET	/api/analytics/dashboard	Private	Summary KPIs, category distribution, 7-day sales trend

4.6 Sample API Responses

Login Response

{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
"user": { "id": "...", "username": "admin", "name": "System Administrator", "role": "admin" } }

Invoice Creation Response

{ "invoice": { "invoice_number": "INV-20260522-001", "customer_name": "Jeevan ",
"subtotal": 900, "gst_amount": 162, "grand_total": 1062, "payment_mode": "UPI" },
"items": [{ "product_name": "Wireless Mouse", "quantity": 2, "gst_rate": 18, ... }] }

5. Source Code Explanation

5.1 Backend server.js (Entry Point)

The Express application is initialised in server.js. It loads environment variables via dotenv, enables CORS for cross-origin requests from the React frontend, registers all route modules under /api/*, and starts the HTTP server on port 5000. Database initialisation runs at startup to connect MongoDB and seed default data if the database is empty.

5.2 Database Configuration config/database.js

All four Mongoose schemas (User, Category, Product, Invoice) are defined here with validation rules and virtual fields. A toJSON/toObject transform remaps MongoDB's _id to id and strips the version key, ensuring clean JSON responses. The initializeDatabase() function handles connection and seeds the admin user (admin/admin123), default categories (Electronics, Groceries, Apparel), and sample products on first run.

Product schema includes a virtual field category_name that returns the populated category name, enabling the frontend to display category labels without extra API calls.

5.3 Auth Middleware middleware/auth.js

A lightweight JWT verification middleware extracts the Bearer token from the Authorization header and verifies it against JWT_SECRET. On success, the decoded user payload is attached to req.user and the request proceeds. On failure, a 401 Unauthorized response is returned immediately. This middleware wraps every private route in the application.

5.4 Billing Route routes/billing.js

Invoice creation is the most complex backend operation. The POST /api/billing/invoice route performs these steps in sequence:

- Validates request payload customer details and at least one item must be present.
- Generates a sequential invoice number in the format INV-YYYYMMDD-XXX by counting today's invoices.
- Stock validation loop each product is fetched and checked for sufficient stock before any mutation.
- Price calculation for each item: gst_amount = unit_price x (gst_rate/100) x quantity; total_price = (unit_price x qty) + gst_amount.
- Invoice document is saved with all computed totals and the embedded items array.
- Stock deduction each product's stock_quantity is decremented using MongoDB's $inc atomic operator.

5.5 Analytics Route routes/analytics.js

The dashboard endpoint uses MongoDB aggregation pipelines to compute KPIs efficiently:

- Overall stats single $group stage sums total_invoices, total_sales, and total_gst across all invoices.
- Profit calculation unwind on items, $lookup to join Product purchase_price, then compute (unit_price - purchase_price) x quantity per line.
- Low-stock count uses $expr with $lte to compare stock_quantity against reorder_level.
- 7-day sales trend matches invoices from last 7 days, groups by date, then fills missing days with zeros server-side.

5.6 Frontend AuthContext.jsx

The React AuthContext stores the JWT token in React state, persists it to localStorage for page refresh, and exposes an authFetch() utility that automatically attaches the Authorization: Bearer <token> header to every API request. On load, the context calls /api/auth/verify to restore the session if a valid token exists in localStorage.

5.7 Frontend App.jsx (Page Router)

App.jsx implements a single-page application router using a currentPage state string. The AppContent component reads auth state — showing Login if unauthenticated, or the main layout (Sidebar + page component) if authenticated. Page switching is managed by passing setCurrentPage down to child components, allowing the Dashboard's low-stock alert to navigate directly to the Inventory page.

6. Application Screenshots

6.1 Login Page

The login page authenticates users with username and password. Default sandbox credentials (admin / admin123) are shown on screen for ease of first-run access. JWT token is issued on successful login.

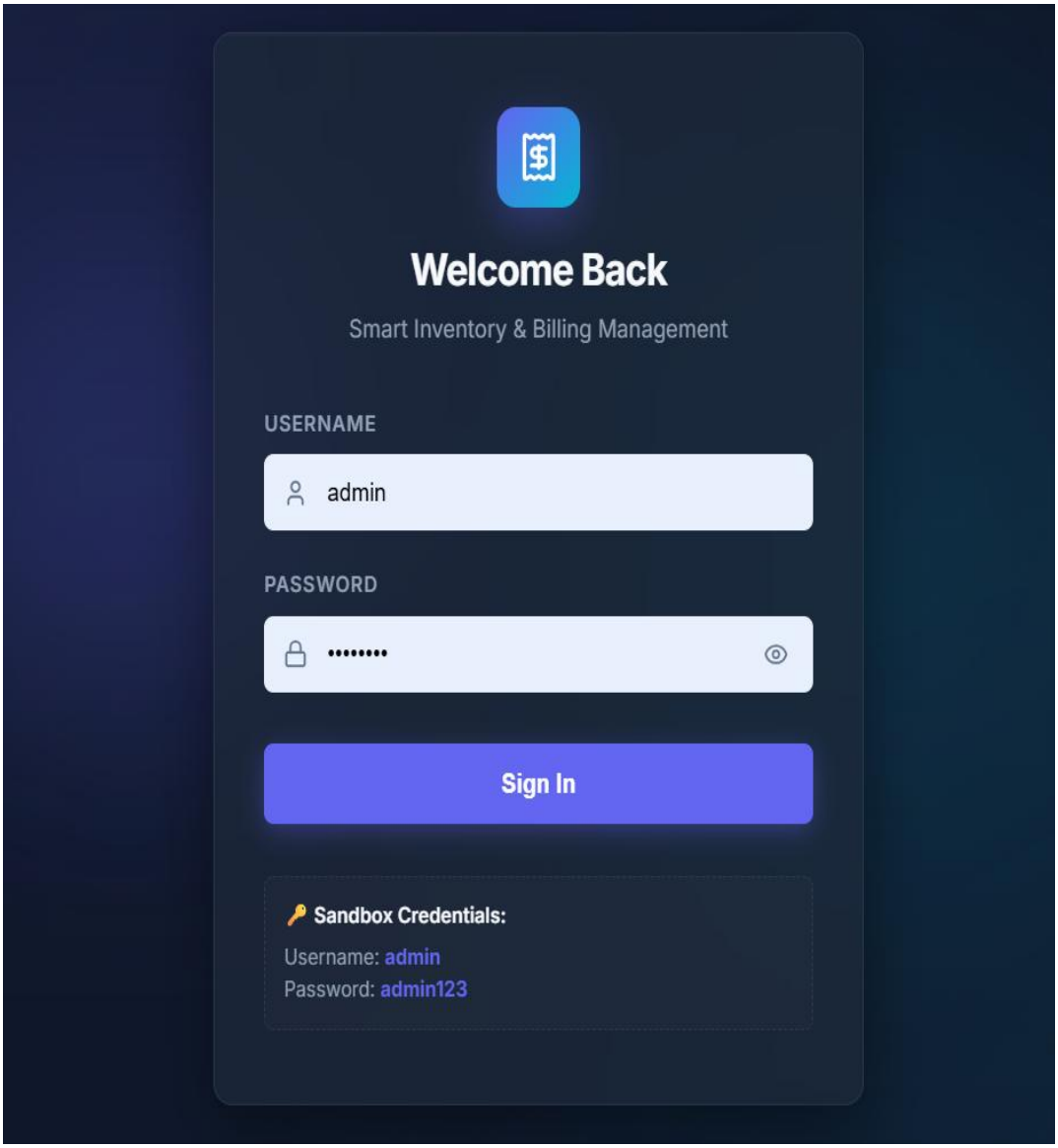


Figure 6.1 — Login page:

6.2 Dashboard Analytics & KPI Overview

The dashboard provides a real-time business summary with four KPI cards (Gross Sales, Net Profit, Total Items, Low Stock Alert), a Sales Revenue Trend chart for the last 7 days, and a Low Stock Alerts panel listing products at or below their reorder level.

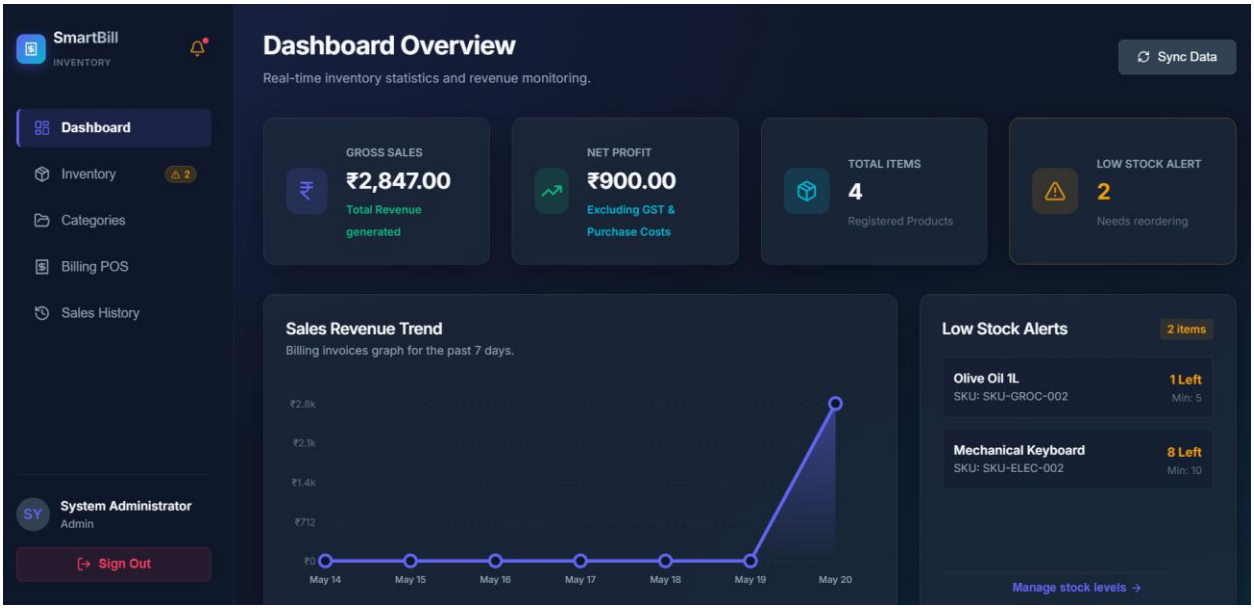


Figure 6.2 — Dashboard: KPI cards, sales trend chart, and low-stock alerts panel

6.3 Inventory Management

The Inventory Stock page lists all products in a table showing SKU, category, stock quantity with colour-coded In Stock / Low Stock badges, safety level, purchase price, selling price, and GST rate. Actions include Add Product, Edit, and Delete. An Export CSV button allows data download.

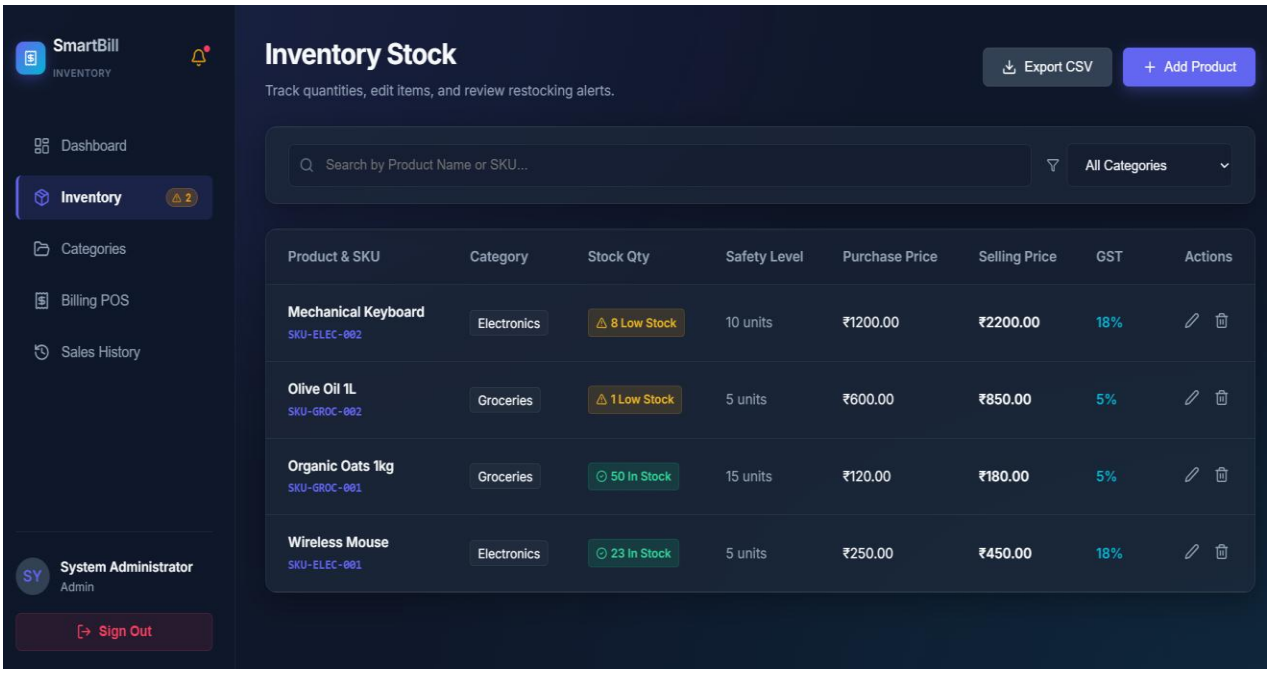


Figure 6.3 — Inventory Stock: product list with stock status badges and action controls

6.4 Category Management

The Category Management page shows all product categories in a table with ID, name, description, and edit/delete actions. New categories can be added via the Add Category button. Deletion is blocked if products exist under that category.

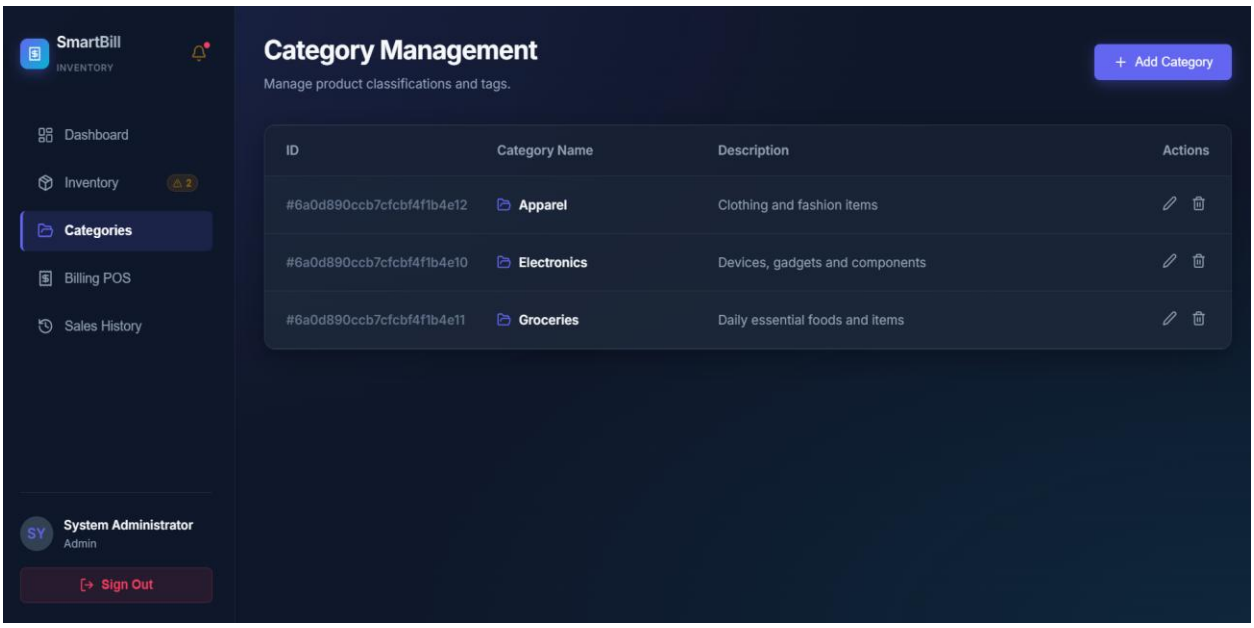


Figure 6.4 — Category Management: category list with CRUD controls

6.5 Point of Sale (POS) Billing Interface

The POS Billing page allows a cashier to search and add products to the cart, enter customer name, phone number, and payment mode (Cash/Card/UPI). The Billing Summary panel auto-calculates subtotal, GST, and Grand Total. Clicking Generate Invoice saves the invoice and deducts stock.

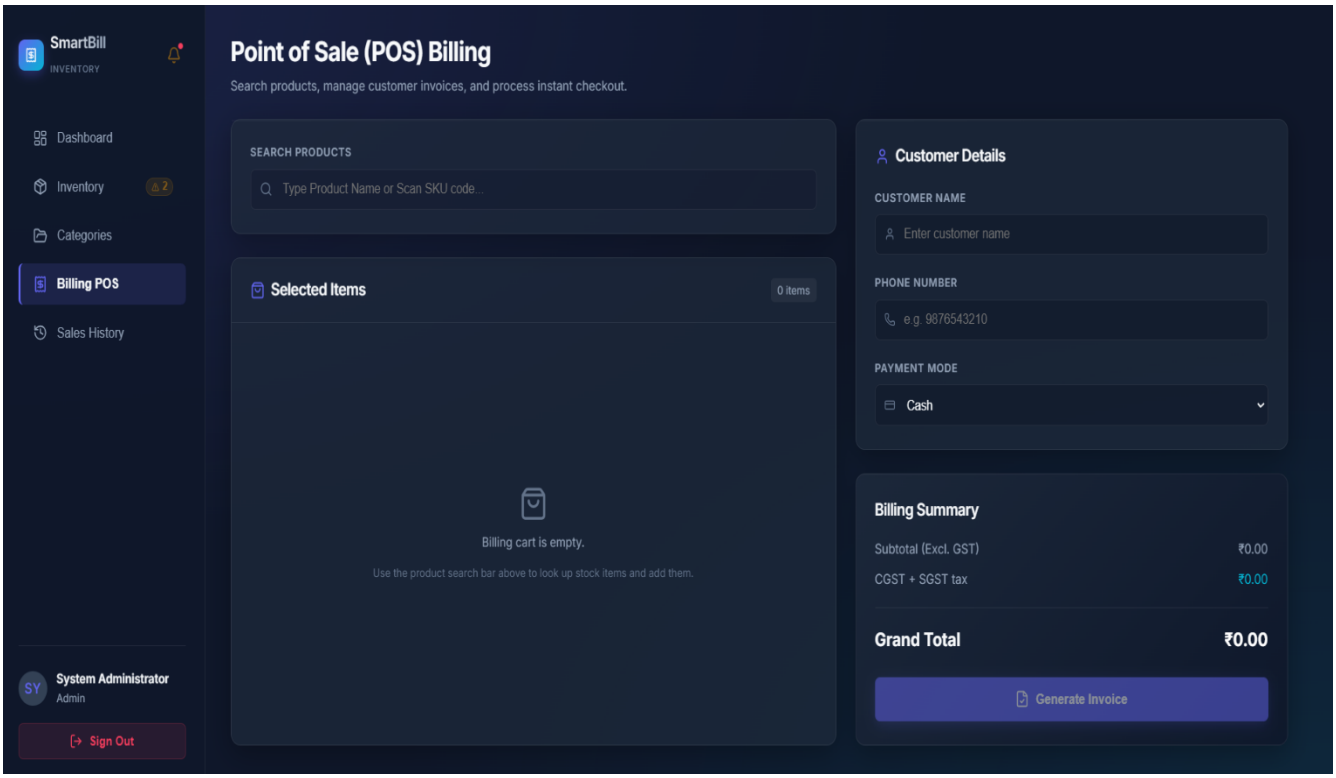


Figure 6.5 — POS Billing: product search, cart, customer details, and billing summary

6.6 Digital Receipt Modal

After invoice generation, a receipt modal is displayed showing the store header, invoice number, date, customer details, itemised line items with GST, subtotal, GST amount, and Grand Total. A Print button triggers the browser's print dialog for physical or PDF receipt output.

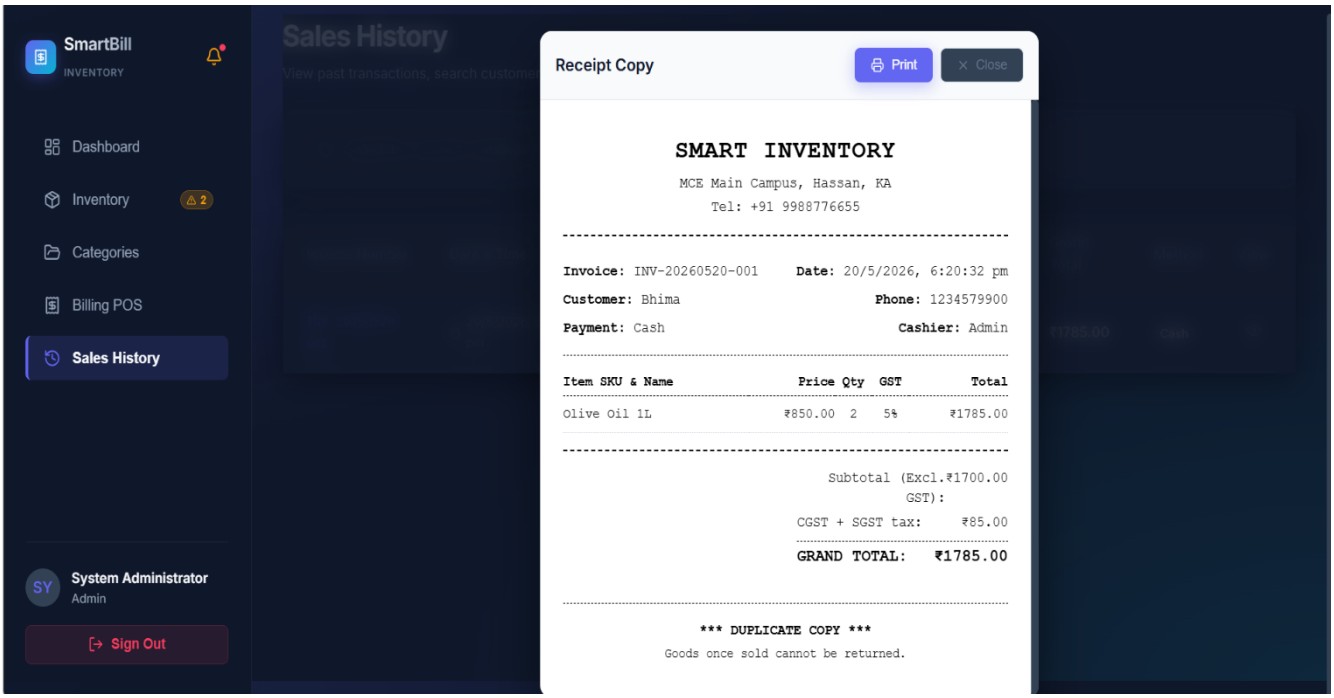


Figure 6.6 Receipt Modal: itemised digital receipt with Print option

6.7 Sales History

The Sales History page lists all past invoices in a table with invoice number, date/time, customer name, phone, subtotal, GST, grand total, and payment mode. A search bar filters by customer name, phone, or invoice number. Each row has a View button to re-open the receipt modal.

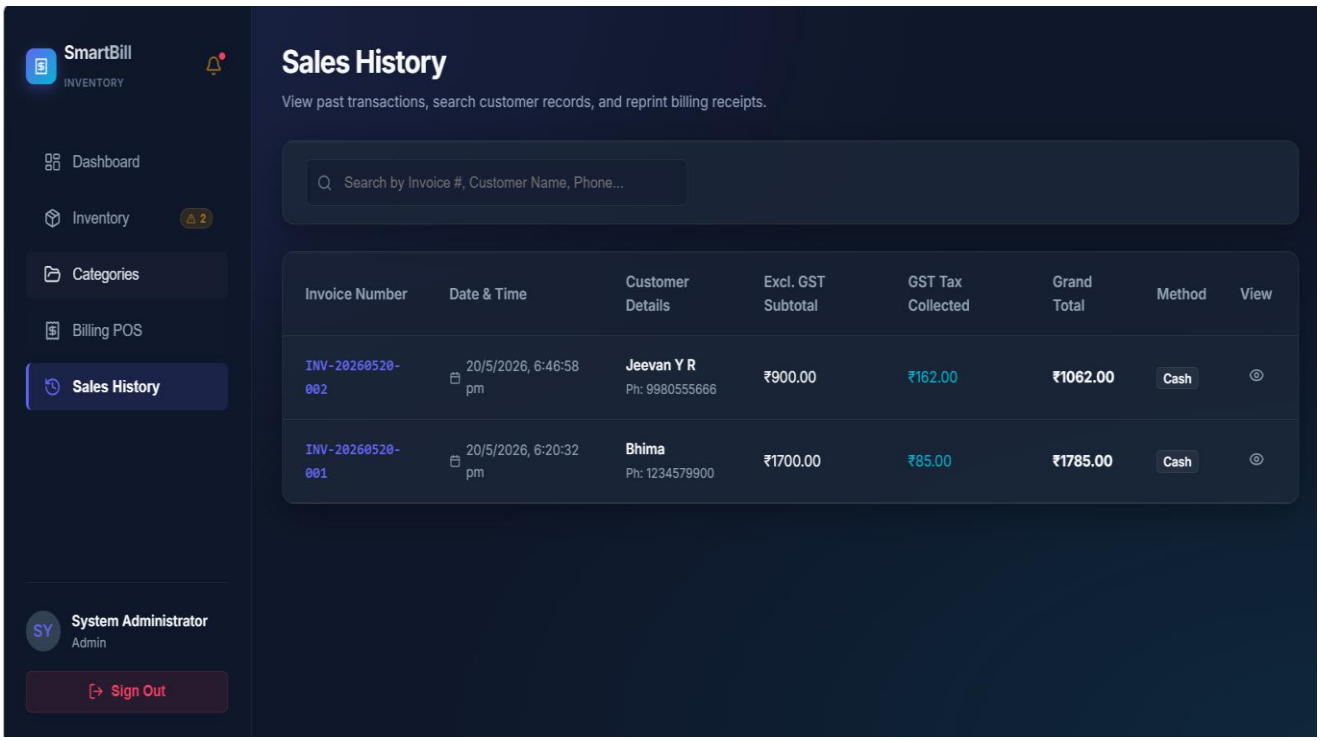


Figure 6.7 — Sales History: all invoices listed with search and view receipt functionality

7. Conclusion

The Smart Inventory and Billing Management System successfully addresses the real-world operational challenges faced by small and medium retail businesses. The system integrates all essential business processes stock management, category organisation, GST-compliant invoicing, and performance analytics into a single cohesive web application.

7.1 Key Achievements

- Implemented a secure JWT-based authentication system with role-aware access control and 24-hour session tokens.
- Built a real-time inventory tracking engine with automatic low-stock detection using configurable reorder levels per product.
- Delivered a fully automated GST billing system supporting multiple tax rates (5%, 12%, 18%, 28%) with sequential invoice number generation.
- Created a comprehensive analytics dashboard powered by MongoDB aggregation pipelines, displaying revenue KPIs, profit margins, category distribution, and a 7-day sales trend chart.
- Designed a responsive dark-themed SPA with intuitive navigation, toast notifications, and a modal receipt system that supports browser print functionality.
- Established a clean RESTful API with 15+ endpoints, consistent error handling, and atomic stock deductions to prevent inventory inconsistencies.

7.2 Technical Highlights

The project demonstrates proficiency across the full MERN stack. MongoDB aggregation pipelines efficiently compute business analytics without loading full datasets into memory. Atomic stock deductions using MongoDB's $inc operator prevent race conditions during concurrent billing. React Context API provides clean separation of authentication state, and the authFetch wrapper eliminates repetitive header injection across all API calls.

7.3 Future Enhancements

- Barcode/QR scanner integration for faster product lookup at the billing counter.
- PDF invoice generation and email dispatch to customers post-purchase.
- Multi-user support with granular role-based permissions (cashier, manager, owner).
- Supplier management module with purchase order tracking.
- GST return summary report generation compatible with government filing formats.
- Mobile-responsive Progressive Web App (PWA) build for tablet-based billing.

7.4 Summary

This project successfully demonstrates how modern web technologies can transform paper-based retail operations into a streamlined, data-driven digital system. The application is production-ready for single-store retail deployments and provides a solid foundation for expansion into a multi-branch, cloud-hosted SaaS product.

References

- MongoDB Documentation https://www.mongodb.com/docs/
- Mongoose ODM Documentation https://mongoosejs.com/docs/
- Express.js Official Guide https://expressjs.com/
- React.js Official Documentation https://react.dev/
- Vite Build Tool https://vitejs.dev/
- JSON Web Tokens (JWT) https://jwt.io/introduction
- bcrypt.js https://www.npmjs.com/package/bcryptjs
- Lucide React Icons https://lucide.dev/
- Central Goods and Services Tax Act, India, 2017